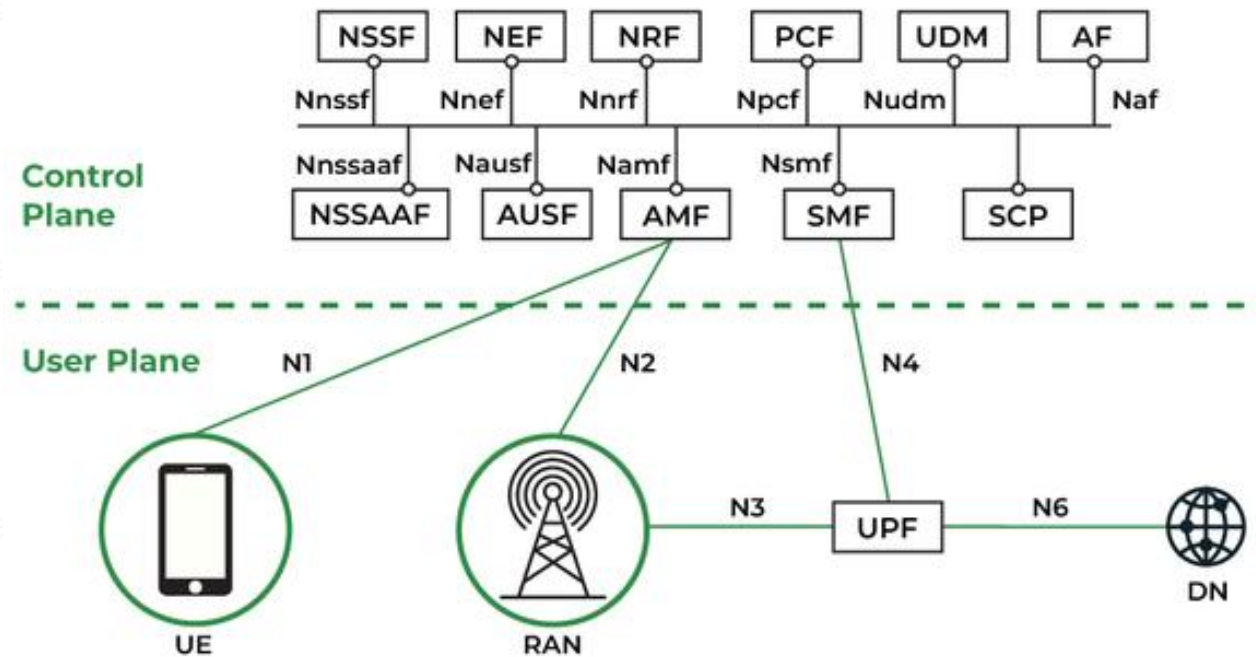


5G Private Network Deployment

FEUP, May/2026

Prof. André Coelho
Prof. Pedro Duarte
Prof. Manuel Ricardo

Introduction



- 5G relies on a **Service-Based Architecture (SBA)**
- **Network Functions (NFs)** act as Service Producers and Service Consumers
- **Control Plane (CP)** and **User Plane (UP)** functions are separated
- Each NF provides services to authorized NFs via **standardized interfaces**
- **Dynamic and flexible** architecture → operators can design, deploy, and adapt NFs as needed

Cellular networks

- Cellular networks **experimentation** historically limited
- High **costs and licensing** barriers
- Open-source and **Software-Defined Radio (SDR)** changed the landscape
- **RF simulators/emulators** enable 5G testing in personal computers



OpenAirInterface (OAI)

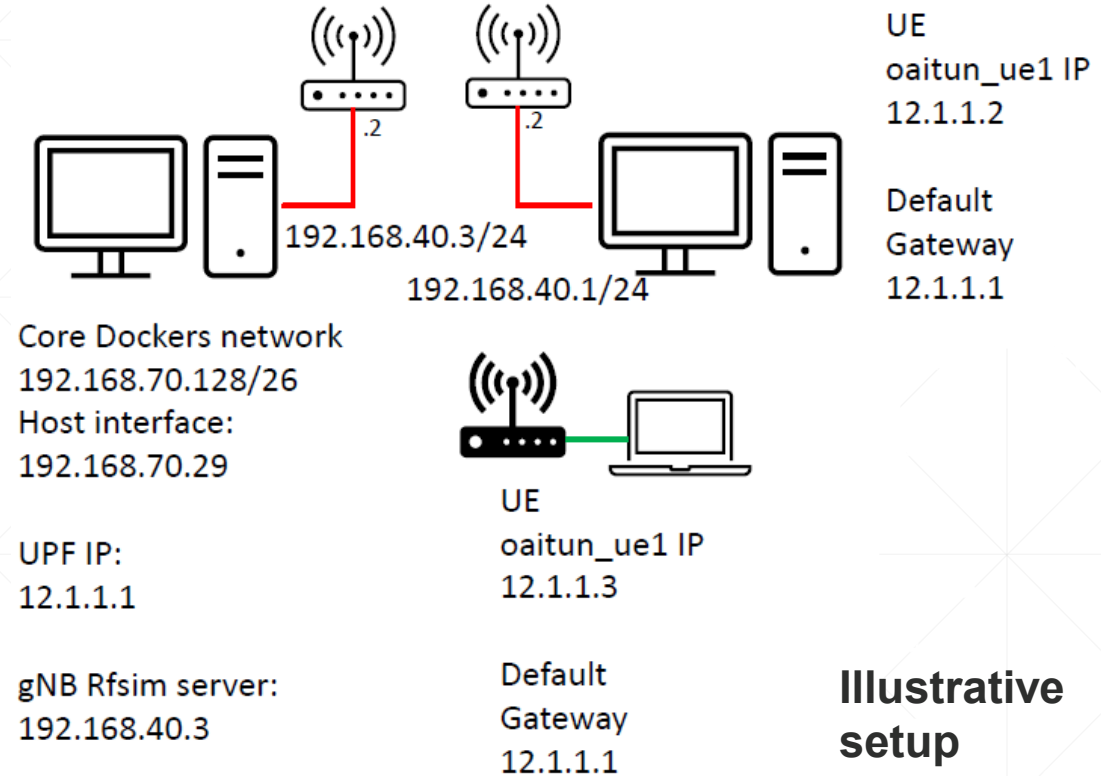
- Leading **open-source 5G framework**
- **5G Core, gNodeB, User Equipment (UE), and RIC (FlexRIC)** implemented as **software** components
- Runs on **x86 computers + SDRs** or **RF simulator/emulator**
- Managed by [OpenAirInterface Software Alliance](#)

OAI overview

- **3GPP-compliant 5G Standalone (SA) architecture**
- Allows to emulate **gNB-UE links in software**
- **Realistic MAC/PHY** layer behavior
- **OAI democratizes 5G experimentation**

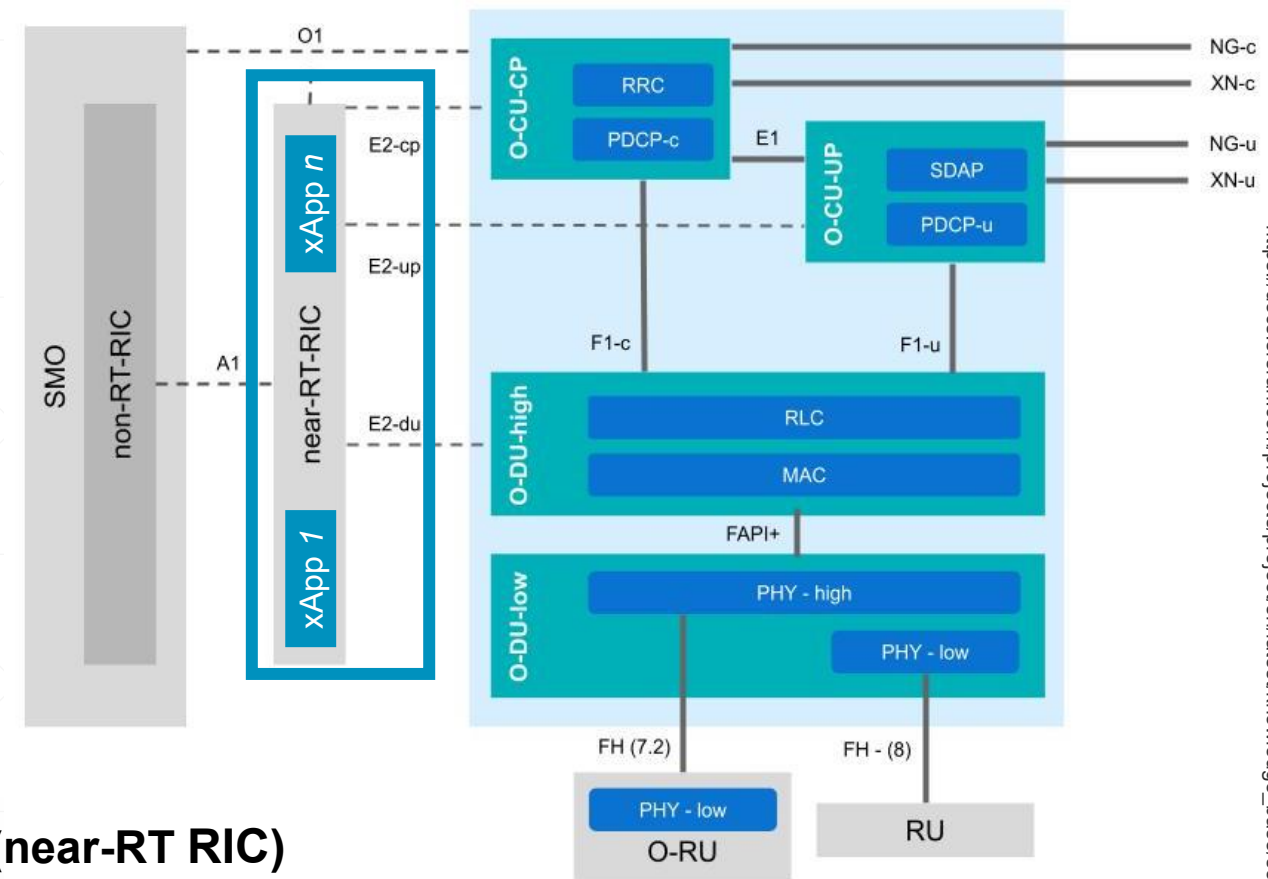
Relevant weblinks

- [OpenAirInterface 5G RAN repository \(GitLab\)](#)
- [OAI 5G Core network overview and repositories](#)
- [OAI tutorial](#) (for RAN, focus on RFsimulator not SDRs)
- [OAI/FlexRIC tutorial](#)
- [OAI documentation hub](#) (various tutorials and guides)



O-RAN

- **O-RAN** splits gNB into **three main functions**
 - **Central Unit (CU)**
 - **Distributed Unit (DU)**
 - **Radio Unit (RU)**
- **Near-Real-Time RAN Intelligent Controller (near-RT RIC)**
 - Hosts xApps → modular software applications
 - Interfaces with RAN via E2 interface
 - Enables dynamic resource management
- For more information, please refer to: <https://doi.org/10.1109/COMST.2023.3239220>



Lab objectives

- Deploy OAI **5G Core Network**, **RAN** (gNB and UE emulation), and **RAN Intelligent Controller (RIC)**
- Enable communications between 5G RAN and Core **across different computers**
- Test end-to-end **connectivity between UE and the Data Network (DN)**
- Integrate and **run custom applications (xApps) to monitor the RAN**
- Change **network configurations**
- Assess **network performance** using Key Performance Metrics

Lab setup

Recommended setup

- **Computer 1** → OAI 5G Core
- **Computer 2** → OAI RAN (gNB + UEs emulation) and FlexRIC (RIC + xApps)
- **Computer 3** → Additional UEs (optional)
- Connections: 5G Core ↔ RAN (via IP network)
- You can change which processes run on different computers, but it is important to have a distributed system (network components running on different computers)
- Install Ubuntu 20.04 or 22.04 on each computer
- **Install dependencies**

```
sudo apt update
```

```
sudo apt install git vim tree net-tools libsctp-dev python3 python3-dev cmake-curses-gui libpcrc2-dev build-essential cmake libfftw3-dev
```

```
sudo apt install libmbdttls-dev libboost-program-options-dev libconfig++-dev libtool autoconf python3-pip curl bison flex iperf unzip automake
```

Prerequisites

- **Install SWIG 4.1**

```
git clone https://github.com/swig/swig.git
```

```
cd swig
```

```
git checkout release-4.1
```

```
./autogen.sh
```

```
./configure --prefix=/usr/
```

```
make -j$(nproc)
```

```
sudo make install
```

- **Check GCC version** – it should be gcc-10, gcc-12, gcc-13

```
gcc --version
```

- If you see that you have gcc-11, consider the steps given [here](#) to switch to a different version (modifying the steps to install either gcc-10, gcc-12, or gcc-13 as the new version)

5G Core network

- **Set up Docker's apt repository (for 5G Core network)**

```
sudo apt-get update
```

```
sudo apt-get install ca-certificates curl
```

```
sudo install -m 0755 -d /etc/apt/keyrings
```

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc
```

```
sudo chmod a+r /etc/apt/keyrings/docker.asc
```

```
echo \
```

```
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.asc] https://download.docker.com/linux/ubuntu \
```

```
$(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}") stable" | \
```

```
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

```
sudo apt-get update
```

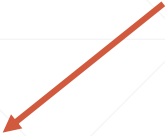
- **Install Docker packages**

```
sudo apt-get install docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
```

- **Verify that the installation is successful**

```
sudo docker run hello-world
```

Execute this
all at once



5G Core network

- **Download and build 5G RAN (file “oai-cn5g.zip” available in Moodle)**

```
cd
```

```
unzip ~/oai-cn5g.zip
```

```
cd ~/oai-cn5g
```

```
sudo docker compose pull
```

- **For starting 5G Core Network**

```
cd ~/oai-cn5g
```

```
sudo docker compose up -d
```

- **For checking containers**

```
sudo docker ps -a
```

- **For stoping 5G Core network**

```
sudo docker compose down
```

5G RAN

- **Download and build 5G RAN (file “openairinterface5g.zip” available in Moodle)**

```
unzip ~/openairinterface5g.zip
```

```
cd ~/oai/cmake_targets/
```

```
sudo chmod +x build_oai
```

```
./build_oai --clean
```

```
./build_oai -l -w SIMU --gNB --nrUE --build-e2 --ninja
```

```
./build_oai --build-lib telnetrv
```

- **Warnings** displayed after the build process can be disregarded if the message “**BUILD SHOULD BE SUCCESSFUL**” appears

FlexRIC

- **Download and build FlexRIC (file “flexric.zip” available in Moodle)**

```
unzip ~/flexric.zip
```

```
cd ~/flexric
```

```
mkdir build
```

```
cd build
```

```
cmake ../
```

```
make -j$(nproc)
```

```
sudo make install
```

Deploy full 5G network

- **Start 5G Core network (terminal 1)**

```
cd ~/oai-cn5g
```

```
sudo docker compose up -d
```

- **Start gNB (terminal 1)**

```
cd ~/oai/cmake_targets/ran_build/build
```

```
sudo ./nr-softmodem -O ../../targets/PROJECTS/GENERIC-NR-5GC/CONF/gnb.sa.band78.fr1.106PRB.usrpb210.conf --gNBs.[0].min_rxtxttime 6 --rfsim --sa
```

- **Start UE (terminal 2)**

```
cd ~/oai/cmake_targets/ran_build/build
```

```
sudo ./nr-uesoftmodem -r 106 --numerology 1 --band 78 -C 3619200000 --rfsim --sa --uicc0.imsi 0010100000000001 --rfsimulator.serveraddr 127.0.0.1
```

- **Start FlexRIC (terminal 3)**

```
cd ~/
```

```
./flexric/build/examples/ric/nearRT-RIC
```

Exchange traffic between network and UE

- **Uplink ping – UE to network (terminal 4)**

ping 192.168.70.135 -I oaitun_ue1

- **Downlink ping – network to UE (terminal 4)**

sudo docker exec -it oai-ext-dn ping <ue_ip>

- **Downlink iPerf**

- **Terminal 4**

- Find out the IP address of the UE by running **ifconfig** and check the IP address field of oaitun_ue1 network Interface
 - Start iPerf server (example for UDP traffic): **iperf -s -u -i 1 -B <ue_ip>**

- **Terminal 5**

- Start iPerf client (example for UDP traffic and bitrate (-b) of 10 Mbit/s):
sudo docker exec -it oai-ext-dn iperf -u -t 100 -i 1 -fk -B 192.168.70.135 -b 10M -c <ue_ip>

- **Uplink iPerf**

- **Terminal 4**

- Initialize iPerf server (example for TCP traffic): **sudo docker exec -it oai-ext-dn iperf -s -i 1 -fk -B 192.168.70.135**

- **Terminal 5**

- Start iPerf client (example for TCP traffic): **iperf -c 192.168.70.135 -i 1 -B <ue_ip>**

Run xApps

- **KPIMON xApp → Key Performance Metrics monitoring**

- Collects RAN metrics and sends them to other xApps, periodically and event-triggered
- Based on E2SM-KPM service model (3GPP TS 28.552). Metrics example: DRB.PdcpSduVolumeDL, DRB.PdcpSduVolumeUL, DRB.RlcSduDelayDI, DRB.UEThpDI, DRB.UEThpUI, RRU.PrbTotDI, RRU.PrbTotUIImplements
- Start (in an independent terminal)
`cd ~/flexric`
`./build/examples/xApp/c/monitor/xapp_kpm_moni`

- **RAN Control (RC) xApp**

- Controls RAN behavior exposed by FlexRIC, supporting
 - REPORT Style 4: Subscription for UE RRC State Changes
 - CONTROL Style 1: Configure QoS flow mappings, create new DRBs, etc → **example action: release a UE's connection**
- Start (in an independent terminal)
`cd ~/flexric`
`./build/examples/xApp/c/kpm_rc/xapp_kpm_rc`

Experiments to be carried out

1. Set up **OAI 5G Core network on one PC**, and **gNB and FlexRIC on another PC**, connected via IP network (4 points)
 - Tip: to run the gNB and Core network on different computers, you need forwarding rules; see the 3rd step within the ["Create the Environment" section](#)
 - Check the current iptables rules: `sudo iptables-save`; Check for DROP rules in raw and filter tables and delete them: `sudo iptables -t raw -F`
`sudo iptables -D DOCKER <no. of rule>`
2. Connect **multiple UEs (e.g., 2)** to gNB and describe the procedure (4 points)
 - Tip: you may need a different namespace per UE
 - Example: [OAI 5G NR SA tutorial to deploy multiple OAI nrUE](#); in point 2 of this webpage, when using RFSimulator, you may not need to include "[0]" in rfsimulator parameters
3. Modify the **carrier frequency** (e.g., 3450 MHz) and **bandwidth** (e.g., 20 or 100 MHz) in the gNB configuration file (4 points)
 - Example: [5G gNB Frequency Configuration](#)
4. Assess **network performance** in terms of throughput (iPerf) and RTT (ping) between UEs and DN (4 points)
 - Plot throughput (TCP and UDP traffic) and RTT measurements for **two different bandwidth** values and **multiple UEs**
5. Develop a simple script/web interface to **monitor and plot metrics** (e.g., throughput) over time **via an xApp** (4 points)
6. **Bonus:** implement and **trigger a gNB action through an xApp** and describe the approach and observations (+2 points, maximum final grade: 20 points)
 - Example: [OAI/FlexRIC tutorial](#)

Submission

- Groups consisting of up to 4 students
- **Zip file** per group containing
 - A **report document** → A4 size, 11pt font, maximum 4 pages
 - Describing the setup, configurations and experiments carried out (previous slide)
 - Presenting screenshots and plots with the results obtained
 - Discussing the main conclusions achieved
 - The **configuration files and scripts** used

Good Work!